



r0cketship

Pre-Flight Hardening Engineering Spec

Discrete one-sprint hardening pass to remove known scaling blockers before Wave-1 ramps.



DOCUMENT

Engineering Spec

DATE

2026-06-01

STATUS

Draft v1 — internal

Why this matters

The r0cketship platform is already shipped and handling traffic. But four known issues will start biting at low-double-digit customer counts:

1. **Stripe subscription invoices are manual.** Every \$4,500/mo **booking** customer needs an operator to confirm payment monthly. At 50 customers = ~2 hours/week of human reconciliation. Breaks at scale.
2. **/signup has no rate-limit or email-verify.** The \$50 signup-bonus is a free-money attack vector. One scripted attack = thousands in fraudulent credits issued.
3. **Footer + email HTML is unsanitized.** Manager-editable HTML fields are XSS-vulnerable if a hostile white-label is provisioned (or compromised).
4. **In-memory pool/predictive/global-counts scans.** Current implementation iterates the leads/persons table in JS — fine at <100k rows per tenant, falls over at 100k+. Several Wave 1 niches (mortgage, ecom, B2B SaaS) will hit this in months, not years.

There's also a 5th smaller win: per-niche dedicated SEO landing pages. The template exists but content per niche is empty — solving this multiplies organic lead flow at near-zero cost.

Sequencing (1–2 sprints, ~10 engineering days)

ORDER	ITEM	DAYS	BLOCKING?	WHY THIS ORDER
1	Stripe subscription auto-charge	3	Hard blocker at >20 paying customers	Without this, \$4,500 customer growth = linear ops cost
2	/signup rate-limit + email-verify	1	Hard blocker before paid ads scale	\$50 bonus is exposed the moment ad spend hits scale
3	DOMPurify on user-editable HTML	1	Hardening (not yet exploited but real)	Cheap to do, removes a future class of incident
4	Per-niche SEO landing pages	2	Not blocking but high-leverage	Each launched niche becomes an SEO asset within 2 weeks
5	SQL push for in-memory scans	3	Soft blocker at ~100k rows/tenant	Lift this BEFORE the niches that will trigger it (mortgage, ecom) hit 100k

Total: 10 days of one engineer, or 5–7 days with two engineers working in parallel.

Recommended parallel split: - Engineer A: items 1, 5 (Stripe auto-charge + SQL push) — both touch billing/queries, same engineer for consistency - Engineer B: items 2, 3, 4 (signup hardening + DOMPurify +

Item 1: Stripe Subscription Auto-Charge

Problem

Currently `5b ZIP subscriptions` records `payments.purpose=subscription` rows manually (operator marks paid → confirm advances `paid_through +1mo`). Stripe topup checkout exists (`createTopup` → Stripe Checkout) but subscription invoicing is not automated. At 50+ recurring customers, this is unsustainable.

Goal

Every `data` , `booking` , and `epartner` subscription auto-renews monthly via Stripe Billing without operator intervention. Failed charges flow into a retry queue + dunning email.

Implementation

- 1. Create Stripe Products + Prices on-the-fly per offer** - When a customer first subscribes to an offer, look up or create: - Stripe Product (one per tenant + offer-type, e.g., `roofers.co:data` , `roofers.co:booking`) - Stripe Price (recurring monthly, in the offer's amount * 100 cents) - Store `stripe_product_id` and `stripe_price_id` on the offer config (new columns on `tenants.offers` JSON or new `offer_stripe_ids` table)
- 2. Migrate `subscribeZip` flow** - Replace the manual-payment flow with Stripe Subscription creation: - `stripe.subscriptions.create({ customer, items: [{ price }], metadata: { tenantId, zipSubscriptionId } })` - Webhook events: - `invoice.paid` → mark `zip_subscriptions.paid_through +1mo`, write `payments.purpose=subscription` with Stripe invoice id - `invoice.payment_failed` → enter dunning state (suspend lead access after 3 failed attempts over 7 days) - `customer.subscription.deleted` → mark ZIP subscription cancelled, free the ZIP for resale
- 3. Provider abstraction stays** - The `PaymentProvider` interface already exists. Add `subscribe(customerId, offer)` and `cancelSubscription(subscriptionId)` methods. Manual provider gets stub that does the current behavior (operator confirmation). Stripe gets the real implementation.
- 4. Dunning email flow** - Day 1 of failure: email customer "Payment failed, try again" - Day 3: email + suspend lead access - Day 7: email "Account closing in 24 hours" + cancel subscription - Day 8: ZIP returned to available pool

Test plan

- Unit test: provider interface stubs work in both manual and Stripe modes
- Integration test (with Stripe test mode):
- `subscribe` → `invoice.paid` webhook → `paid_through` advances
- `subscribe` → `invoice.payment_failed` → suspend + dunning email

- cancel subscription → ZIP returns to pool
- Manual QA: create test customer, subscribe to \$1,500/mo with Stripe test card, advance the clock with Stripe time-machine, verify renewal works

Files to touch

- `src/billing/providers/stripe.ts` — add subscription methods
- `src/billing/providers/manual.ts` — stub subscription methods (existing behavior)
- `src/billing/subscriptions.ts` (or wherever `subscribeZip` lives) — route through provider
- `src/api/webhooks/stripe/[tenant]/route.ts` — handle subscription events
- New table `subscription_dunning` (optional but recommended for audit)

Estimated effort: 3 engineering days

Item 2: `/signup` Rate-Limit + Email-Verify

Problem

`/signup` creates a customer + auto-grants `$50 wallet credit` with no rate-limit or email-verify. A scripted attacker can: 1. Generate 1,000 disposable emails (mailinator, plus-addressing) 2. POST 1,000 signups in a minute 3. Drain \$50,000 in fake wallet credits 4. Resell or use them to buy real leads from your pool

Goal

Make the \$50 bonus require: 1. Real email verified before issuance 2. IP-based rate limiting (≤ 3 signups per IP per 24h) 3. Disposable email detection (block known mailinator/tempmail domains)

Implementation

- 1. Email verification gate** - `POST /api/signup` creates customer with `email_verified=false` and `signup_bonus_pending=50` - Send verification email with one-time link `/verify?token=<jwt>` - On click, set `email_verified=true` AND grant the \$50 (write `credit_ledger` entry with type=signup_bonus) - Bonus only granted at verification, not at signup
- 2. IP rate-limit** - Use a simple in-memory + DB hybrid: track `(ip, hour_bucket)` count - New table `signup_attempts (ip, ts)` with index on ip - Reject after 3 attempts per 24h with friendly "Please contact support" message
- 3. Disposable email detection** - Maintain a list of known disposable email domains (use the `disposable-email-domains` npm package, ~1MB list, refreshed monthly) - Reject signups from those domains at the API layer - Plus-addressing (`name+tag@gmail.com`) is fine — let Gmail handle that
- 4. Bot detection (optional but recommended)** - Add Cloudflare Turnstile or hCaptcha on the signup form - Server-side verify token before creating account - Skip for users with valid auth session (e.g., agency partner

inviting clients)

Test plan

- Unit test: rate-limiter blocks 4th attempt from same IP
- Unit test: disposable domain rejected
- Integration test: signup → email verify → bonus granted
- Manual: attempt 5 signups from same IP, verify 4th is rejected
- Manual: signup with mailinator.com, verify rejection
- Manual: signup with valid email, do NOT click verify link, verify bonus NOT granted

Files to touch

- `src/api/signup/route.ts` — add rate-limit + verify gate
- `src/auth/email-verify.ts` — new module
- `src/billing/wallet.ts` — `grantSignupBonus(customerId)` called only on verify
- New table `signup_attempts`
- New table `email_verify_tokens` (or use signed JWTs)

Estimated effort: 1 engineering day

Item 3: DOMPurify on User-Editable HTML

Problem

Several fields are HTML-editable by manager-role users: - `tenants.footer_html` (custom footer) - `customer_integrations.emailBodyHtml` (per-customer email template)

Currently these are stored and rendered as raw HTML. If a hostile white-label is provisioned (or a manager account is compromised), they can inject scripts that execute in any customer's browser.

Goal

All user-editable HTML is sanitized at write time AND at render time. XSS attack surface = zero.

Implementation

1. **Sanitize at write** - Install `isomorphic-dompurify` (~30KB, runs in both Node and browser) - In server actions that update HTML fields, run `DOMPurify.sanitize(html, { ALLOWED_TAGS: [...], ALLOWED_ATTR: [...] })` before persisting - Allowed tags: `p`, `br`, `strong`, `em`, `a`, `ul`, `ol`, `li`, `h1` - `h6`, `img`, `div`, `span`, `blockquote` - Allowed attrs: `href`, `src`, `alt`, `title`, `class` (whitelisted classes only) - Block: `script`, `iframe`, `object`, `embed`, `form`, `input`, `style`, `link`, event handlers (`onclick`, etc.)

2. Sanitize at render (defense in depth) - Wherever these fields are rendered (footer, email body), run through DOMPurify again - This is belt-and-suspenders — if old unsanitized content is in the DB, the render-time pass catches it

3. Migration - One-off script: pull all `tenants.footer_html` and `customer_integrations.emailBodyHtml` rows, run through sanitizer, write back - Confirm no functional regressions (sanitizer should be permissive enough that normal-looking HTML is untouched)

Test plan

- Unit test: `<script>alert(1)</script>` → stripped
- Unit test: `<a href="javascript:..."` → href stripped
- Unit test: `<img src=x onerror=alert(1)>` → onerror stripped
- Unit test: normal HTML (`<p><strong>Hello</strong> <a href="https://...">link</a></p>`) → preserved
- Manual: try to inject a script via `/admin/branding` , verify it's stripped on save

Files to touch

- `src/lib/html-sanitize.ts` — new shared module
- Server actions that write HTML fields (find via grep for `footer_html` and `emailBodyHtml`)
- Render sites where HTML is `dangerouslySetInnerHTML` 'd

Estimated effort: 1 engineering day

Item 4: Per-Niche SEO Landing Pages

Problem

The platform supports `/niches` (master directory) and per-tenant marketing pages. But there's no SEO-optimized landing page per (niche × geography). This is huge: organic search for "[niche] leads in [city]" can drive 30–40% of signups at zero ad cost once it ranks.

Goal

For every active tenant, dynamically generate SEO landing pages at `/[niche-slug]-leads-in-[zip-or-city]` that: - Rank for the target keyword in 30–90 days - Drive organic signups from intent search traffic - Reinforce the niche brand authority via volume of indexed pages

Implementation

1. Route + data - New route: `/app/seo/[niche]/[location]/page.tsx` - Niche slug pulled from tenant config (e.g., "commercial-roofing", "solar", "mortgage") - Location: ZIP code OR city name, dynamically generated from your geography list (e.g., top 500 US cities + top 1000 ZIPs) - Page content templated from tenant marketing content + location-specific dynamic copy

2. Content template - H1: "[Niche] Leads in [City] — ZIP-Exclusive, DNC-Clean" - Intro: 200 words niche-specific copy - Section 2: "Why [Niche] businesses in [City] use rocketship" — local stats, common pain points - Section 3: "How our [Niche] feed works in [City]" — 3-step explainer - Section 4: "ZIP coverage in [City]" — list of ZIPs in this metro - Section 5: Trust signals (compliance, integrations) - Section 6: CTA — "Get 50 free [niche] leads in [city]" - Footer: same as marketing footer

3. Sitemap auto-include - Update `app/sitemap.ts` to include all (niche x location) combinations for active tenants - 12 niches x 500 cities = 6,000 indexed pages per niche brand - This is the entire moat — competitors can't replicate this volume without our backend

4. Schema markup - Add `LocalBusiness` JSON-LD per location page - Add `Service` schema (niche-specific) - Add `FAQPage` schema with 3–5 niche-specific FAQs

5. Content uniqueness - Avoid thin-content / doorway-page penalty: each page needs ≥500 words of substantively-different copy - Use the tenant's existing `marketingContent.subhead` + city-specific stats (population, business density) pulled from DataForSEO - Optional: GPT-4 / Claude API for per-page intro generation (cache forever to avoid recompute cost)

Test plan

- Manual: visit `solarsignal.com/solar-leads-in-90210` — page renders with correct location data
- Manual: sitemap.xml at solarsignal.com includes all generated URLs
- Manual: Google Search Console — submit sitemap, monitor index coverage at +14 days
- Lighthouse: SEO score ≥95 per page (mobile-friendly, structured data, etc.)

Files to touch

- `app/seo/[niche]/[location]/page.tsx` — new route
- `app/sitemap.ts` — extend
- `src/marketing/seo-content.ts` — content templates
- `src/lib/locations.ts` — city + ZIP catalog (one-time data load)
- Optionally: `src/lib/ai-content.ts` for GPT/Claude content generation

Estimated effort: 2 engineering days

Item 5: SQL Push for In-Memory Scans

Problem

Several hot paths iterate the leads/persons table in-memory in JavaScript: - Lead pool filtering (`/leads`) - Predictive scoring (computing scores for the entire tenant pool) - Global counts (`/admin/insights`)

This works at <100k rows. At 100k–1M rows per tenant, response time degrades from <100ms to multi-second. Several Wave 1 niches will hit this in months.

Goal

Push all heavy filtering, scoring, and aggregation into Postgres so response times stay <100ms at 1M+ rows per tenant.

Implementation

- 1. Lead pool filtering** - Current: SELECT all leads → filter in JS - New: SQL query with WHERE clauses + LIMIT/OFFSET pagination - Add indexes: `(tenantId, zip, segment, age_tier, predictive_score DESC)` covering index - Use Drizzle ORM query builders (project already uses Drizzle)
- 2. Predictive scoring** - Current: `predictiveScore()` runs per lead in JS, then collected - New: SQL-computed score (materialized as a column updated on lead-ingest, refreshed nightly via cron) - Use a Postgres materialized view OR computed column - The scoring formula (intent tier + cross-site converted +30 + recency + commercial) is pure math — fits in SQL
- 3. Global counts** - Current: JS aggregation - New: SQL `COUNT() GROUP BY tenantId, zip, segment, age_tier` + caching (5-minute Redis cache acceptable) - For `/admin/insights` cross-tenant: same SQL but no tenant filter
- 4. Bulk ingest optimization** - Current: row-by-row upsert (mentioned in roadmap as deferred) - New: Postgres `INSERT ... ON CONFLICT DO UPDATE` with multi-row batching (1,000 rows per batch) - 10–100x faster for large CSV uploads

Test plan

- Performance: seed 100k, 1M, 10M rows; measure `/leads` filter response time before/after
- Correctness: same query result set against in-memory vs SQL implementations
- Regression: existing 164 tests must remain green

Files to touch

- `src/leads/pool.ts` (or wherever filtering happens)
- `src/predictive/scoring.ts` — move math to SQL view
- `src/predictive/analytics.ts` — replace JS aggregation with SQL
- `src/ingest/upsert.ts` — bulk insert with ON CONFLICT
- Migration: new indexes + materialized view + computed columns

Estimated effort: 3 engineering days

Acceptance criteria (whole spec)

- [] All 5 items deployed live on Vultr
- [] All 164 existing tests still pass
- [] At least 30 new tests added (auto-charge webhooks, rate-limit, sanitizer, SEO render, SQL push)

- [] Manual QA on each item completed and documented
- [] No regressions in `/admin` workflows
- [] Performance regression test on `/leads` filter at 100k rows passes <200ms

Out of scope (defer to future sprints)

- Stripe subscription auto-charge with usage-based billing (overage credits) — Tier 2 future feature
- SAML / SSO for enterprise customers — Tier 3 future
- Multi-region data residency — only if we get an enterprise customer needing it
- Customer-facing reporting API (we have it internally — only expose if customer asks)
- A/B testing infrastructure for landing pages — defer to Wave 3 when we have data on what works